

Singleton Design Pattern Explained

Which response do you prefer? Responses may take a moment to load.



Response 1

Sure! Here's a version of the **Singleton pattern in C++** that uses **manual locking with `std::mutex`**. This is useful if you're working in **pre-C++11 environments** or want **fine control** over synchronization.

Thread-Safe Singleton with Manual Locking

```
cpp

#include <iostream>
#include <thread>
#include <mutex>

class Singleton {
private:
    static Singleton* instance;
    static std::mutex mutex_;

    Singleton() {
        std::cout << "Singleton instance created.\n";
    }

public:
    // Thread-safe getInstance using mutex
    static Singleton* getInstance() {
        std::lock_guard<std::mutex> lock(mutex_); // Lock for thread safety
        if (instance == nullptr) {
            instance = new Singleton();
        }
        return instance;
    }

    void showMessage(const std::string& threadName) {
        std::cout << "Accessed by " << threadName << " - Address: " << this << "\n";
    }

    // Prevent copying
    Singleton(const Singleton&) = delete;
    Singleton& operator=(const Singleton&) = delete;
};

// Initialize static members
Singleton* Singleton::instance = nullptr;
std::mutex Singleton::mutex_;

// Function for threads to access Singleton
void accessSingleton(const std::string& threadName) {
    Singleton* s = Singleton::getInstance();
    s->showMessage(threadName);
}

int main() {
    std::thread t1(accessSingleton, "Thread 1");
    std::thread t2(accessSingleton, "Thread 2");
    std::thread t3(accessSingleton, "Thread 3");
    std::thread t4(accessSingleton, "Thread 4");

    t1.join();
    t2.join();
    t3.join();
}
```

```
t4.join();

return 0;
}
```

Explanation

- ``std::mutex mutex_``: Ensures only one thread can enter the critical section at a time.
- ``std::lock_guard<std::mutex> lock(mutex_)``: Locks the mutex when entering ``getInstance()`` and automatically unlocks when it exits (RAII).
- All threads share the same ``Singleton* instance``.

Note on Performance

This method **locks every time** ``getInstance()`` is called, even after the singleton has been created. If you want to improve performance, consider the **double-checked locking** pattern. Want to see that version too?

I prefer this response



Response 2

Sure! Here's a **manual locking version** of the Singleton design pattern using **mutexes**, which you can use if you're working with **pre-C++11** or want full control over thread synchronization.

Manual Locking Singleton (Thread-safe for pre-C++11)

```
cpp

#include <iostream>
#include <thread>
#include <mutex>

class Singleton {
private:
    static Singleton* instance;
    static std::mutex mtx;

    // Private constructor
    Singleton() {
        std::cout << "Singleton instance created.\n";
    }

public:
    // Static method with locking to ensure only one instance is created
    static Singleton* getInstance() {
        std::lock_guard<std::mutex> lock(mtx); // Lock mutex to make thread-safe
        if (instance == nullptr) {
            instance = new Singleton();
        }
        return instance;
    }

    void showMessage(const std::string& threadName) {
        std::cout << "Accessed by " << threadName << " - Address: " << this << "\n";
    }
}
```

```

// Prevent copy and assignment
Singleton(const Singleton&) = delete;
Singleton& operator=(const Singleton&) = delete;
};

// Initialize static members
Singleton* Singleton::instance = nullptr;
std::mutex Singleton::mtx;

// Function for each thread to call
void accessSingleton(const std::string& threadName) {
    Singleton* s = Singleton::getInstance();
    s->showMessage(threadName);
}

int main() {
    // Create multiple threads
    std::thread t1(accessSingleton, "Thread 1");
    std::thread t2(accessSingleton, "Thread 2");
    std::thread t3(accessSingleton, "Thread 3");
    std::thread t4(accessSingleton, "Thread 4");

    // Wait for threads to finish
    t1.join();
    t2.join();
    t3.join();
    t4.join();

    return 0;
}

```

Sample Output

text

```

`Singleton instance created.
Accessed by Thread 2 - Address: 0x563cbaf00160
Accessed by Thread 1 - Address: 0x563cbaf00160
Accessed by Thread 4 - Address: 0x563cbaf00160
Accessed by Thread 3 - Address: 0x563cbaf00160
`

```

- The ``std::lock_guard<std::mutex>`` ensures that only one thread can enter the critical section at a time.
- The Singleton instance is created only once, and all threads use the same address.

Notes:

- This approach is **safe** but might be **less performant** than the C++11 static-local-variable method due to the locking on every access.
- If performance matters, consider **Double-Checked Locking** (let me know if you want that version too).

Would you like to see the **Double-Checked Locking** version as well?

I prefer this response